

Lista 07 – Heaps

May 15, 2026

PARA CADA QUESTÃO, ESCREVA O ALGORITMO EM PSEUDOCÓDIGO, JUSTIFIQUE AS DECISÕES DE PROJETO E INDIQUE O TEMPO DE EXECUÇÃO DE CADA SOLUÇÃO.

- ① Trace a construção do heap máximo **bottom-up** para a lista

$$V = [4, 10, 3, 5, 1, 8, 7, 2, 9, 6].$$

Mostre o estado do vetor após cada chamada de **Desce**. Ao final, desenhe a árvore correspondente ao heap construído.

- ② Trace a execução do **HeapSort** para a lista

$$V = [3, 1, 4, 1, 5, 9, 2, 6].$$

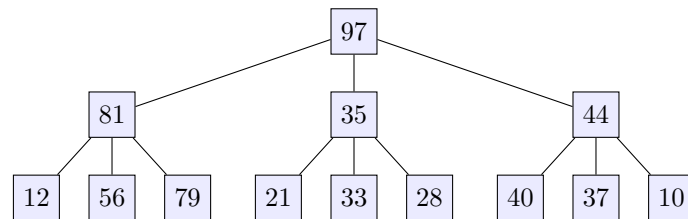
Mostre o estado do vetor após cada extração do máximo. Indique em cada passo qual parte do vetor ainda é heap e qual parte já está ordenada.

- ③ Mostre que a construção bottom-up do heap executa em $O(n)$, calculando explicitamente a soma

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot h.$$

Dica: use a identidade $\sum_{h=0}^{\infty} \frac{h}{2^h} = 2$.

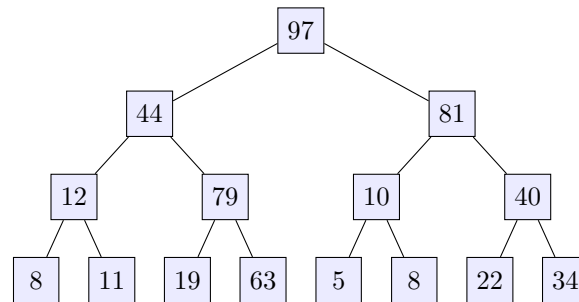
- ④ A estrutura de dados **Tri-Heap-Max** é uma variante do Heap-Max onde cada elemento possui **3 filhos**, ao invés de apenas 2.



No Tri-Heap-Max, o filho do elemento na posição k está nas posições $3k - 1$, $3k$ e $3k + 1$.

- Implemente as operações **insert(x)** e **removeMax()** no Tri-Heap-Max.
- Analise cuidadosamente o número de **comparações** realizadas por essas duas operações, e compare com as operações correspondentes no Heap-Max convencional. A variante com 3 filhos é mais eficiente?

- ⑤ Imagine a ideia de manter o Heap-Max **arrumadinho**: os dois filhos de cada elemento são mantidos em ordem, com o maior sempre do lado direito e o menor do lado esquerdo.



A ideia é que, ao reorganizar o heap após uma inserção ou remoção, já sabemos que o maior filho está sempre à direita economizando uma comparação por nível.

- Implemente as operações **insert(x)** e **removeMax()** no Heap-Max-arrumadinho.
 - Estime o tempo de execução das suas implementações. A economia de comparações compensa o custo extra de manter a ordem entre os filhos?
- ⑥ Implemente uma fila de prioridade que suporte também a operação

AumentaChave(i, k) : aumenta o valor do elemento na posição i para k .

- Escreva o pseudocódigo da operação.
 - Qual é a complexidade da operação? Justifique.
 - Como você modificaria a operação para suportar também **DiminuiChave**(i, k)?
- ⑦ Você é responsável por monitorar o tráfego de dados em uma rede de servidores em tempo real. A cada segundo, um novo registro de tamanho de pacote (em bytes) é gerado, e você precisa manter a **mediana** atualizada para identificar anomalias.

Como os dados chegam em fluxo contínuo, a estrutura deve suportar:

- Inserção** de um novo valor: complexidade $O(\log n)$.
 - Consulta da mediana**: complexidade $O(1)$.
- Descreva como construir a estrutura usando um **heap máximo** e um **heap mínimo** simultaneamente. Explique como a mediana pode ser consultada em $O(1)$ a partir dessa estrutura.
 - Escreva as funções de inserção (*sift-up*) e extração (*sift-down*) em um **heap mínimo**.
 - Escreva o procedimento completo de **inserção de um novo pacote** na estrutura dupla. Certifique-se de manter o invariante que garante a consulta em $O(1)$.

⑧ Um **heap d -ário** é uma generalização onde cada nó tem até d filhos (em vez de 2).

- a) Qual é a altura de um heap d -ário com n elementos, em função de d e n ?
- b) Como isso afeta as complexidades de **insert** e **removeMax**? Existe um valor ótimo de d ?
- c) Para $d = 4$, escreva a fórmula que dá as posições dos filhos do elemento na posição k , e a posição do pai.